

Asymptotic

Target geometry, certificate refinement,
and flat-sector precision

Additional findings beyond the existing code reviews

Prepared for	Vladimir Reshetnikov
Repository	VladimirReshetnikov/Asymptotic
Snapshot	921387e5ba1239bfda96e63e64e89bf63d9c41e6
Declared package	AsymptoticInverse 1.8.0; Wolfram Language 15.0+
Evidence	Source inspection and independent exact-arithmetic models

Principal result. Three concrete implementation issues remain outside the existing consolidated review register: incorrect target approaches inferred by composite arithmetic, a certificate-refinement controller that can freeze arithmetic precision after success, and premature loss of exponential grading in flat products. A fourth finding identifies a conservative but unnecessarily restrictive sign test for powers of Gamma/Barnes inverse observables.

Execution boundary. Twenty independent Python model tests passed. The native Wolfram service returned connection errors; the upstream package, its native suite, and the supplied Wolfram proposals were not executed in this audit. No native benchmark or fully validated patch release is claimed.

Contents

1	Scope, snapshot, and the non-duplication baseline	1
1.1	What was inspected and what was executed	1
1.2	Findings register	2
2	What the package adds to Wolfram Language	2
2.1	The useful abstraction is a mathematical contract, not just a formula	2
2.2	Native functionality should not be understated	3
2.3	Capability comparison	3
2.4	The package already builds on the native system	4
2.5	Concrete paired experiments to run	4
3	Mathematical checks on specialized designs	5
3.1	Why exact Lambert cores are useful	5
3.2	An independently checkable Dirichlet tail	5
3.3	Fourier coefficients and differentiation	6
4	F01: composite arithmetic can use the wrong target approach	6
4.1	The heuristic and the missing check	6
4.2	Witness A: a flat inverse with a pole target	7
4.3	Witness B: the negative-source Erfc tail	8
4.4	Witness C: a downward quadratic threshold	8
4.5	Repair at the producers, with checked legacy recovery	8
4.6	Acceptance tests and metamorphic coverage	9
5	F02: certificate success can freeze precision refinement	9
5.1	The proof already established by each successful attempt	10
5.2	The controller's two independent tasks are conflated	10
5.3	A restricted exact-rational experiment	10
5.4	The stored intersection already proves a better error bound	11
5.5	A controller that responds to the limiting error source	12
5.6	Why the existing tests do not exercise this case	13
6	F03: flat products forget the dominant exponential grade	13
6.1	Two independent truncations must remain independent	13
6.2	A one-sector example loses three algebraic orders	13
6.3	The loss grows with retained depth	14

6.4	A graded error semiring sufficient for this repair	15
6.5	Avoid computing discarded coefficient polynomials	16
6.6	The delivered candidate and its validation boundary	16
7	F04: a positive observable can have a negative source	17
7.1	Do not repair this by multiplying exponents blindly	17
7.2	Inherited precision remains a constraint on source replay	18
8	A focused development plan, without duplicating the old roadmap	18
8.1	Change set A: establish target geometry before arithmetic	18
8.2	Change set B: separate certification from accuracy completion	19
8.3	Change set C: retain grades in flat product error scheduling	19
8.4	Change set D: support reflected positive observables	19
8.5	New cross-family invariants worth institutionalizing	19
9	Overall assessment	20
A	Reproducibility and delivered artifacts	20
A.1	Executed evidence	20
A.2	Native experiments supplied but not executed	21
A.3	Proposal scope	22
A.4	Rebuilding the article	22
B	Coverage and novelty ledger	22
B.1	Direct source coverage	22
B.2	Relationship to earlier review entries	23
B.3	Remaining uncertainty	23

1 Scope, snapshot, and the non-duplication baseline

This report is a *delta audit*: it examines additional failure modes rather than recasting the repository’s existing reviews as new discoveries. The inspected snapshot is 921387e5ba1239bfda96e63e64e89bf63d9c41e6. The repository identifies the package as AsymptoticInverse 1.8.0 with a Wolfram Language 15.0 minimum. The modular kernel is the maintained implementation; the root standalone file is a generated distribution. These are snapshot statements, not assertions about a subsequently updated branch.[1, 2, 3]

The anti-duplication baseline is the consolidated `CODE_REVIEW_STATUS.md`, the code-review index, and the detailed accompanying summaries for reviews 6–9. The consolidated register accounts for 87 numbered findings as 32 grouped work items, followed by 12 extension directions. I did not independently read every page of all nine review articles. Consequently, the claim here is that the new findings do not occur in the *consolidated register or the reviewed summaries*; that is a more precise claim than an exhaustive textual non-overlap guarantee.[3, 4, 5, 6, 7, 8]

Two prominent earlier findings are already addressed in this snapshot: bounded optional dense `SeriesData` construction, and a shared rejection of unsupported real fractional powers of pure remainders. They are not new findings below. Similarly, this report does not re-recommend general assumption capture, fixed-margin recipe refinement, global resource budgets, a common scale protocol, logarithmic-tail export repair, generic native CI, or a broad multiscale-transseries redesign. Those subjects already have entries in the register.[3]

1.1 What was inspected and what was executed

Seventeen implementation files were inspected directly, with focused coverage of ordinary construction, precision arithmetic, certificates, source/target transformations, Gamma/Barnes operations, Fourier coefficients, native special-function import, flat sectors, and arithmetic fallback. The precise file/range ledger is supplied in `evidence/source_manifest.json`. Two native test files were read in full where they directly bear on the new findings. This is not a claim of exhaustive line-by-line verification of every module.[9, 10, 11, 16, 26, 27]

The investigation used pinned GitHub connector reads. A complete local checkout was not obtained. The Wolfram connector returned HTTP 502 errors, including for a version-only probe; no native package execution succeeded. The delivered Wolfram runners therefore record *prospective* experiments, not completed ones. The article distinguishes three evidence classes:

Source deduction.

A consequence of inspected definitions and dispatch, with the relevant constructors checked for the metadata on which the deduction depends.

Independent mathematical/model verification.

A derivation or a restricted Python implementation using exact rational arithmetic. It can validate a theorem, counterexample mechanism, or control policy without establishing native Wolfram evaluation behavior.

Native characterization required.

A supplied call whose result should be recorded in a fresh compatible kernel. No result from such a call is invented here.

The twenty executed tests include exact target-orientation cases, 500 deterministic randomized outward-rounding checks within one test, rational interval containment checks, flat inverse coefficients, and

graded error-envelope comparisons. They are not twenty upstream package tests. Their complete log and generated evidence are included.

1.2 Findings register

ID	Priority	Finding	Evidence / consequence
F01	High	Composite fallback can infer the wrong endpoint or approach side for flat pole targets, negative Erfc tails, and downward quadratic thresholds.	Source deduction; exact geometry; wrong germ metadata or false incompatibility.
F02	Medium	Successful-but-insufficient certificates can keep a fixed enclosure order; the reported error also ignores a sharper bound already implied by the retained root interval.	Source deduction; exact rational controller model; failure to reach attainable accuracy.
F03	Medium	Flat multiplication forgets exponential grades before combining tail bounds and computes discarded coefficient products unnecessarily.	Source deduction; exact flat-coefficient and envelope models; avoidably weak bounds and extra algebra.
F04	Low / feature	Gamma/Barnes power replay tests the source sign rather than the sign of the already represented observable.	Source deduction; valid positive even-power observables are conservatively rejected.

F02 and F03 are not accusations of false mathematical certificates or understated remainders. In the examined cases, they are respectively a precision-controller defect and an unnecessarily pessimistic, potentially expensive error transport. F04 is an explicit scope restriction that can be generalized safely; it is not a wrong-result bug.

2 What the package adds to Wolfram Language

2.1 The useful abstraction is a mathematical contract, not just a formula

The package’s main abstraction is a finite expression together with a branch, a scale, and a remainder. Ordinary power–log jets have the form

$$\sum_{\alpha \in S, \alpha < P} u^\alpha P_\alpha(\log u) + \mathcal{O}\left(u^P(1 + |\log u|)^D\right), \quad u \rightarrow 0^+, \quad (1)$$

where S is a finite retained support and the coefficients are polynomials in the logarithm. The implementation separates exact weight comparison, coefficient canonicalization, sparse convolution, inverse-coefficient construction, and precision propagation. The same public object can also represent other scales, but its metadata conventions vary by family.[9, 11]

An expression alone cannot encode whether a source tends to 0^+ or $+\infty$, whether an inverse is the lower or upper real branch, whether a flat zero sector is exact, or whether differentiation of the unknown remainder is justified. Those distinctions are the reason a specialized package can remain useful even

when the native system can obtain many of its finite expressions. They are also the source of the cross-family defects in this audit. The distinction between an asymptotic relation and a differentiable asymptotic relation is standard and important.[39]

2.2 Native functionality should not be understated

Asymptotic is not confined to Taylor polynomials. Its documented interface covers finite and infinite expansion points, generalized asymptotic scales, and branch/parameter options. Its order and term-goal conventions must be distinguished from the package’s exclusive exponent cutoff. The native default for **Assumptions** is `$Assumptions`; generating all relevant conditions can require an explicit option. A comparison that uses identical integer arguments without inspecting the achieved expansion is not an equal-work comparison.[28]

AsymptoticSolve is the native counterpart to *implicit* asymptotic inversion, rather than **InverseSeries** alone. It accepts equations or systems, can specify a limiting solution point, and supports a real-domain argument. This makes it the appropriate baseline for $f(x) = y$ with branch conditions. A failed bounded invocation would establish only the behavior of that particular invocation and kernel, not a universal native limitation.[29]

InverseSeries handles ramified examples as well as ordinary locally linear reversion; its documentation includes inversion of series whose first nonzero term is quadratic. It would be incorrect to advertise all Puiseux inversion as a feature absent from Wolfram Language. **ComposeSeries** supplies the related native composition operation.[30, 31]

A single **SeriesData** uses a rational exponent lattice described by integer indices and a denominator. That is not the same as a restriction on every expression produced by Wolfram Language: logarithmic coefficients and external special-function or exponential factors may surround a native series. The package’s direct sparse representation of exact irrational supports is therefore a representation advantage, not proof that the entire native language cannot manipulate the same mathematical functions.[32, 33]

2.3 Capability comparison

Task	Native baseline	Package distinction at the snapshot
Ordinary and ramified reversion	Series , InverseSeries , AsymptoticSolve .	Explicit real source approach, transported input precision, residual metadata, and multiple coefficient-construction routes.
Irrational power-log support	Generalized native asymptotics and implicit solving; one SeriesData remains rational-lattice based.	Sparse exact weights and weight-based exclusive cutoffs are first-class rather than incidental expression structure.
Lambert-normalized inverses	ProductLog already supplies Lambert branches and numerical evaluation.	Retains an exact Lambert core and expands corrections around it with an explicit inverse-family contract.
Special functions	Broad native Series/Asymptotic coverage.	Specialized adapters and a native-series importer attach real-domain and absolute-remainder information.

Task	Native baseline	Package distinction at the snapshot
Flat exponentials	Native expressions can retain exponentials outside algebraic series; task-specific transformations are possible.	An exact zero sector, inclusive exponential depth, exclusive inner-power cutoff, and separate inner/tail errors.
Logarithmic oscillations	Native trigonometric/exponential algebra and asymptotics.	Finite Fourier-polynomial coefficient algebra with exact frequencies, conjugacy checks, and explicit mode budgets.
Quantitative inverse validation	<code>Interval</code> and general symbolic/numeric tools are available.	A specialized certificate object for a root of a stored explicit equation using rational intervals and derivative/residual bounds.
Integration and summation	Native <code>AsymptoticIntegrate</code> and <code>AsymptoticSum</code> .	Not a broad replacement for these native tasks; the existing extension register already covers related future work.

The comparison above combines the documented native interfaces with the inspected package modules; it is not a completed benchmark.[28, 29, 30, 32, 34, 35, 36, 37, 2, 15, 22, 23, 10, 3]

2.4 The package already builds on the native system

`NativeSpecialFunctions.wl` calls native `Series` with `Analytic->False`, then walks the structured result while transporting absolute errors. It keeps distinct exponential carriers or oscillatory components in separate envelopes when a single ordered coefficient algebra is unsuitable. Its `SeriesData` tree case deliberately sacrifices half a lattice step rather than assuming an unknown logarithmic tail has degree zero. That is materially different from the older ordinary native-tail concern in the consolidated register. A blanket statement that every native import path discards logarithmic tail information would miss this distinction.[23, 3, 33]

This layered design is appropriate: let native special-function knowledge supply structured local data, then apply a narrower, inspectable remainder calculus. The key obligation is to transport the *entire germ*, including its target approach, across that layer. F01 shows what happens when the expression and bound survive but the approach is guessed incorrectly.

2.5 Concrete paired experiments to run

The archive contains `code/CompareBuiltins.wls`, with paired calls for polynomial reversion, ramified reversion, irrational implicit inversion, an `Erfc` tail, large-order zeta, and a Gamma inverse. Representative calls are:

```

InverseSeries[Series[x + x^2, {x, 0, 5}], y]
AsymptoticInverse[x + x^2, {x, 0}, {y, 6}]

AsymptoticSolve[x + x^Sqrt[2] == y,
  {x, 0}, {y, 0, 4}, Reals]
AsymptoticInverse[x + x^Sqrt[2], {x, 0}, {y, 3}]

```

The first pair is an ordinary sanity check. The second is a characterization pair, not an assertion that its two order specifications are equivalent. Normalize the source branch, finite expression, first omitted scale, and parameter assumptions before evaluating relative performance. The supplied runner records results and messages, not a fabricated success/failure matrix.

3 Mathematical checks on specialized designs

These checks provide context for the new findings without claiming a proof of every implementation path.

3.1 Why exact Lambert cores are useful

For the inverse of a large positive logarithmic Gamma value T , the leading equation is

$$X(\log X - 1) = T, \quad X = \frac{T}{W(T/e)}.$$

The standard Stirling terms give

$$\log \Gamma(X) = T - \frac{1}{2} \log X + \frac{1}{2} \log(2\pi) + \mathcal{O}(X^{-1}).$$

Since the derivative of the core is $\log X$, the first source correction is

$$x - X = \frac{1}{2} - \frac{\log(2\pi)}{2 \log X} + \text{smaller terms}.$$

The inspected Gamma recurrence uses precisely the useful separation between powers of $1/X$ and a reciprocal logarithmic coefficient variable. The core is an exact **ProductLog** expression; the Stirling expansion is still Poincaré data, not a convergent exact forward series.[19, 34, 40]

For Barnes $G(1 + X)$, the leading equation is

$$\frac{X^2}{2} \left(\log X - \frac{3}{2} \right) = T, \quad X = \sqrt{\frac{4T}{W(4T/e^3)}}.$$

Its derivative is $X(\log X - 1)$. This explains the different reciprocal-log variable used by the Barnes recurrence and the necessary argument shift. These identities follow directly by substitution; they are not native numerical observations.[20, 41]

3.2 An independently checkable Dirichlet tail

For real $S > 1$, the tail after retaining $1, 2^{-S}, \dots, (m-1)^{-S}$ in $\zeta(S)$ obeys

$$m^{-S} \leq \sum_{n=m}^{\infty} n^{-S} \leq m^{-S} + \int_m^{\infty} t^{-S} dt = m^{-S} \left(1 + \frac{m}{S-1} \right). \quad (2)$$

Thus $w = e^{-S}$ makes the terms $w^{\log n}$, with an explicit first-neglected-term comparison. This is a good example of a scale choice whose value lies in an auditable support and tail, rather than in claiming that native zeta asymptotics are absent. The inspected Dirichlet adapter follows this structure.[24]

3.3 Fourier coefficients and differentiation

A coefficient mode $P(\ell)e^{i\omega\ell}$, with $\ell = \log u$, is acted on by the Euler operator as

$$u \frac{d}{du} (P(\ell)e^{i\omega\ell}) = (P'(\ell) + i\omega P(\ell))e^{i\omega\ell}.$$

The Fourier module uses this operator and verifies the conjugacy relation between opposite frequencies to establish real-valued coefficients. Oscillatory zeros are not used as nonvanishing error scales; polynomial envelopes bound amplitudes instead. These are important distinctions from a naive algebra of trigonometric leading coefficients.[22]

The remainder-differentiation restrictions in the ordinary and flat calculi are also deliberate. A bare $\mathcal{O}(R)$ is not automatically differentiable with the derivative of R as an error scale. The flat module records a specific analytic contract and uses its phase-dependent derivative loss. F03 preserves that distinction; its proposed product is not permission to differentiate arbitrary unknown errors.[11, 15, 16, 39]

4 F01: composite arithmetic can use the wrong target approach

Classification: high-priority correctness issue in germ metadata. The finite coefficients need not be wrong for this to matter. A valid remainder at $y \rightarrow +\infty$ is not a valid remainder contract at $y \rightarrow 0^+$. The composite fallback can make exactly this substitution of approaches.

Source locus. `AsymptoticInverse/Kernel/SeriesEnvelopeArithmetic.wl: seriesEnvelopeApproach, seriesEnvelopeData, and seriesEnvelopeBinary`. The relevant producers are `flatConstruct`, `flatOpsMake`, `specialErfc`, and `specialThreshold`. [17, 15, 16, 18].

4.1 The heuristic and the missing check

The fallback first accepts a stored `SeriesApproach`. Otherwise it searches for an expansion point or `Limit`; for a flat representation it uses `TargetOffset`, and for an original flat inverse it uses the model's `Offset`. It attempts recovery from a positive small coordinate only when this search leaves the endpoint unknown. If the direction is still unknown, it gives `TargetScale` priority over the leading/core coefficient and infers the side from that sign.[17]

Those fields do not all have the semantics required by that inference. A monomial target *offset* is not necessarily the target *limit*; an outer affine scale does not by itself determine the approach side of a non-monotone or reflected adapter. Moreover, once these shortcuts produce an endpoint and a direction, the code does not verify that the recorded small coordinate actually tends to zero there. The operand's domain is checked for contradiction, but that is weaker than checking that it contains an eventual neighborhood along the claimed approach.[17]

The distinction can be written precisely. Suppose an object records a positive coordinate $u(y)$ and an approach $\mathcal{A}$. Required invariants include

$$u(y) > 0 \text{ eventually along } \mathcal{A}, \quad \lim_{y \rightarrow \mathcal{A}} u(y) = 0, \quad \text{the retained domain holds eventually along } \mathcal{A}. \quad (3)$$

A domain such as $y > 0$ is compatible with both $y \rightarrow 0^+$ and $y \rightarrow +\infty$; non-contradiction alone cannot choose between them.

4.2 Witness A: a flat inverse with a pole target

Consider the supported exact model

$$f(x) = \frac{1}{x} + e^{-1/x}, \quad x \rightarrow 0^+. \quad (4)$$

Its positive monomial core coordinate is $u = x$, its core power is $q = -1$, its coefficient is $a = 1$, and its offset is $b = 0$. The target therefore tends to $+\infty$, not to 0. The constructor call is

```
s = AsymptoticFlatInverse[
  1/x + Exp[-1/x], {x, 0}, {y, 1}];
```

The flat constructor stores its core model and coordinate but no **Limit** or **SeriesApproach**. A derived flat object similarly stores its **FlatRepresentation** without an explicit target approach. The fallback consequently chooses the offset 0 and the positive-coefficient side, namely $y \rightarrow 0^+$. This is a direct source deduction, not a recorded native transcript.[15, 16, 17]

The discrepancy is not merely formal. Let $t = 1/x > 0$. Then

$$y = t + e^{-t}, \quad \frac{dy}{dt} = 1 - e^{-t} > 0.$$

The image of this branch is $(1, \infty)$ because $t + e^{-t} \rightarrow 1$ as $t \rightarrow 0^+$ and tends to infinity as $t \rightarrow \infty$. There is *no* positive-source root for $0 < y < 1$. In particular, interpreting this inverse as a germ near 0^+ cannot be justified by the original equation.

The actual large-target expansion starts with

$$t = y - e^{-y} + \dots, \quad x = \frac{1}{y} + \frac{e^{-y}}{y^2} + \dots.$$

Here the positive core inverse coordinate is $u_0 = 1/y$, which tends to zero precisely at the large positive target end. Checking (3) would immediately reject the guessed endpoint.

A white-box characterization isolates the problem from unrelated dispatch:

```
AsymptoticInverse[Private[seriesEnvelopeApproach[
  s[[1]], 20000]
(* Desired: Point -> Infinity, Direction -> "FromBelow" *)
```

The public expression **SeriesAdd[s, Exp[-y]]** is also included in the supplied runner. The extra exponential is not a finite power-log coefficient in the flat monomial coordinate; the flat-specific path is expected to fall back to composite arithmetic. The runner records whether that public path produces an object or a failure, separately from the directly established inference defect. No exact public output is asserted without native execution.

The upstream flat-operation tests already construct (4) to check differentiation with a negative leading power. Thus this is not a request to support a previously unsupported source family; it concerns transport of an already supported family into the fallback arithmetic.[26]

4.3 Witness B: the negative-source Erfc tail

Write an affine Erfc target as

$$y = b + s \operatorname{erfc}(x), \quad s \neq 0.$$

For $x \rightarrow -\infty$, set $z = -x \rightarrow +\infty$. Using $\operatorname{erfc}(-z) = 2 - \operatorname{erfc}(z)$ gives

$$y - (b + 2s) = -s \operatorname{erfc}(z). \quad (5)$$

Since the positive tail is positive, the target approaches $b + 2s$ *from below* when $s > 0$ and from above when $s < 0$. This follows directly from the reflection identity and is consistent with the tail model used by the adapter.[18, 42]

For default $b = 0, s = 1$, the call

```
s = AsymptoticSpecialInverse[
  "Erfc", {x, -Infinity}, {y, 2}];
```

records `Limit->2`, `SourceSign->-1`, and `TargetScale->1`. The fallback's scale-sign shortcut therefore chooses `FromAbove`, whereas the branch requires `FromBelow`. The exact coefficient used in the side test must incorporate the source-tail reflection, not just the outer target scale.[18, 17]

The positive-tail domain in this adapter can even make the claimed opposite-side approach have no eventual points satisfying the domain. A later symbolic calculation may then fail or reason under an empty asymptotic approach. The audit does not claim to have observed a particular unsound simplification downstream; the incorrectly inferred side is itself the defect that must be repaired first.

4.4 Witness C: a downward quadratic threshold

For

$$y = b + sa(x - x_0)^2,$$

the target-side sign is $\operatorname{sign}(sa)$, independently of which real source side is selected. In particular,

```
s = AsymptoticSpecialInverse[
  "QuadraticThreshold", {x, 0}, {y, 2},
  "QuadraticCoefficient" -> -1];
```

represents a target tending to 0 from below. The adapter retains the negative leading coefficient of the complete quadratic, but it also stores the default `TargetScale->1`. The fallback gives the latter priority and infers the opposite side.[18, 9, 17]

The three witnesses are one root cause with three different manifestations: offset is confused with endpoint, affine scale with reflected tail orientation, and affine scale with total curvature. Fixing only the negative flat exponent case would leave the other two intact.

4.5 Repair at the producers, with checked legacy recovery

For the exact chart

$$y = b + au^q, \quad u \rightarrow 0^+, \quad aq \neq 0, \quad (6)$$

the target approach is determined without a symbolic limit:

Condition	Point	Direction
$q > 0, a > 0$	b	FromAbove
$q > 0, a < 0$	b	FromBelow
$q < 0, a > 0$	$+\infty$	FromBelow
$q < 0, a < 0$	$-\infty$	FromAbove

For the Erfc adapter, multiply the sign of the outer target scale by `SourceSign`. For a quadratic threshold, use the coefficient of the *complete* quadratic. These computations should populate `SeriesApproach` when the object is created. Flat-derived operations should carry the same chart-derived approach forward because multiplication, truncation, and differentiation do not change the independent-variable germ.

Legacy recovery must not let an unverified `Offset` suppress recovery from the actual small coordinate. A sensible order is: accept producer-established approach evidence; otherwise recover from an invertible recorded coordinate; otherwise use a heuristic only after proving (3). If the proof is unavailable, return an explicit uncertainty failure rather than an invented approach. This is a concrete repair of F01, not a re-proposal of the general scale protocol already listed in the earlier reviews.

The supplied `ExpectedTargetApproach` and `AttachExpectedApproach` helpers implement the explicit formulas for these three families without overriding upstream definitions. They return new objects and intentionally reject unsupported chart families. They are not a complete migration layer: operations which subsequently build a new object still need producer-level propagation or a fresh explicit attachment. The Wolfram helpers require native validation.

4.6 Acceptance tests and metamorphic coverage

The independent geometry model contains sixteen cases: eight monomial charts, four Erfc orientations, and four quadratic scale/curvature combinations. The inspected shortcuts disagree with exact target geometry in eight cases. This count describes a deliberately constructed model grid, not an estimated failure rate for real workloads.

The production regressions should check original and derived flat objects, both signs of q and a , both Erfc source infinities and affine target signs, and both quadratic curvature signs. An affine metamorphic check is particularly effective: for a nonzero target multiplier c , $y \mapsto d + cy$ preserves the target side when $c > 0$ and reverses it when $c < 0$. Compare producer metadata, fallback metadata, and direct small-coordinate limits after the same transformation. Finally, exercise at least one public mixed-scale arithmetic path; a private helper regression alone would miss a producer that supplied stale overriding metadata.

5 F02: certificate success can freeze precision refinement

Classification: medium-priority accuracy-controller defect, not a false certificate. The inspected exact interval proof is structurally sound for the supported expression class. The problem is that a successful proof of *some* error bound can prevent the controller from increasing the arithmetic precision needed to prove the *requested* error bound.

Source locus. `AsymptoticInverse/Kernel/InverseCertificates.wl`: `certRound`, `certAttempt`, and the public `InverseCertificate` refinement loop.[10].

5.1 The proof already established by each successful attempt

Let $I = [a, b]$ be a source interval, let $c \in (a, b)$ be a rational center, and suppose a structurally evaluated derivative enclosure D has one sign and satisfies $|f'| \geq \mu > 0$ on I . Let

$$f(c) - y \in R = [r_-, r_+], \quad \varepsilon = \max(|r_-|, |r_+|).$$

The mean-value radius is $r = \varepsilon/\mu$. If $[c - r, c + r] \subseteq I$, continuity and monotonicity establish a unique root there. When that containment fails, the implementation can instead prove opposite endpoint signs. Once existence is known, the mean-value identity also yields

$$x_* \in J = I \cap [c - r, c + r] \cap \left(c - \frac{R}{D}\right). \quad (7)$$

The division is interval division with outward rational rounding. The implementation stores this sharper intersection as `RootEnclosure`. Decimal arithmetic chooses a seed; the successful proof uses rational endpoints and explicit elementary-function tail bounds.[10]

This is a useful trust boundary. Merely obtaining a high-precision `FindRoot` value would not supply the same proof object. Conversely, nothing here implies that native Wolfram interval facilities cannot be used to build similar proofs; the distinction is the package's specialized retained certificate and equation provenance.[35, 10]

5.2 The controller's two independent tasks are conflated

The loop has to solve two different problems: establish a root enclosure at all, and make the center-error bound sufficiently small. Its source structure is essentially:

```
result = certAttempt[... , order, ...];
If[certificateSucceeded && requestedAccuracyReached, return];
If[certificateSucceeded && centerWasAutomatic,
  interval = result["RootEnclosure"];
  center = Mean[interval];
  optionallyRefineTheAsymptoticSeed[],
  order = Min[2000, 2 order]
];
```

Thus an automatic-center attempt that succeeds but misses the accuracy goal narrows the source interval and changes the center, yet retains the same enclosure order. Arithmetic precision rises only in the other branch. Each attempt can continue to certify a true but inadequate bound at the same rounding resolution. Increasing `MaxRefinements` repeats the wrong type of work.[10]

The initial-order heuristic compounds this for relative-only requests: the decimal-digit estimate reads `TargetError` but not `RelativeError`. With no absolute tolerance, a very small relative tolerance does not affect the starting order. An adaptive controller could compensate for that omission, but the successful-attempt branch is precisely where compensation can be suppressed.[10]

5.3 A restricted exact-rational experiment

The supplied Python model specializes the relevant arithmetic to

$$f(x) = x^2, \quad y = 2, \quad I_0 = [1, 2].$$

It implements the inspected dyadic outward-rounding formula, the relevant interval products, the derivative/residual proof, and the intersection (7). A 90-digit decimal approximation supplies an exact rational initial center. This seed is an experimental choice, not a claim about the native seed generated by the package. Every resulting root interval is independently checked by the exact inequalities

$$0 < J_- \quad \text{and} \quad J_-^2 \leq 2 \leq J_+^2.$$

Set the initial enclosure order to 2, so that the inspected bit rule gives $4(2 + 10) = 48$ significant bits, and request absolute error 10^{-60} . The baseline scheduling model performs 65 successful attempts without raising the order. Its final reported radius is exactly

$$r_{64} = \frac{1}{199032864766430} \approx 5.02429586778809 \times 10^{-15}. \quad (8)$$

The requested accuracy is not reached. Under a simple alternative policy that doubles the order after every unsuccessful accuracy test, the same model reaches its goal in seven attempts, at order 128 and 552 bits. The final reported radius is approximately 9.9643×10^{-91} .

These are exact-model results, not native package timings or native return values. The comparison establishes a concrete arithmetic/refinement mechanism and a repair direction; the delivered native characterization call separately tests the actual package:

```
s = AsymptoticInverse[x^2, {x, 0}, {y, 2}];
InverseCertificate[s, 2,
  "Interval" -> {1, 2},
  "TargetError" -> 10^-60,
  "EnclosureOrder" -> 2,
  "MaxRefinements" -> 64,
  "RefineExpansion" -> False]
```

The expected source-derived symptom is an unmet accuracy request with unchanged enclosure order after successful automatic-center attempts. The exact native history remains to be measured.

5.4 The stored intersection already proves a better error bound

The controller also discards useful information when it reports only $r = \varepsilon/\mu$ as **CertifiedErrorBound**. Since $x_* \in J$, an additional rigorous upper bound is

$$B_J = \max\{|J_- - c|, |J_+ - c|\}, \quad B = \min\{\varepsilon/\mu, B_J\}. \quad (9)$$

No new symbolic enclosure or function evaluation is required. A corresponding lower bound is the maximum of the existing residual-based lower bound and the distance from c to J .

In the last baseline model attempt above, the root intersection proves

$$B_J = \frac{1}{5192296858534827628530496329220096} \approx 1.92593 \times 10^{-34},$$

while the reported radius remains approximately 5.0×10^{-15} . The code has already proved about nineteen more decimal orders of center accuracy than the controller uses. This particular sharper bound still does not reach 10^{-60} , so both issues matter: tightening the reported bound is not a substitute for raising arithmetic precision.

Proposition 1. *If a successful certificate establishes $x_* \in J$, replacing its center-error upper bound by (9) preserves certification. Raising the lower bound by the distance from c to J also preserves certification.*

Proof. The continuous function $x \mapsto |x - c|$ attains its maximum over an interval at an endpoint and its minimum at c if $c \in J$, otherwise at the nearest endpoint. Both interval-distance bounds therefore hold for every $x_* \in J$. Intersecting them with previously proved bounds cannot remove the actual error. $\square$

The supplied `TightenCertificate` helper performs this post-processing on an already successful exact-rational certificate. It does not certify a new root or enclose an unspecified input-remainder family. It preserves the original equation and source-domain provenance.

5.5 A controller that responds to the limiting error source

A minimal safe repair is to raise the enclosure order after a successful attempt that misses its goal and fails a progress test. A better policy distinguishes the following limiting quantities: center displacement, width of the residual enclosure at a point, width of the derivative enclosure, and width of the verified root interval.

If the center is poor but arithmetic uncertainty is small, changing the center is productive. If the residual enclosure straddles zero largely because of rounding or elementary-function tail error, recomputing at higher enclosure order is productive. If the derivative is near zero or has severe interval dependency, source-interval narrowing can be productive. These actions are not mutually exclusive; the controller should not encode certificate success as a reason never to raise arithmetic precision.

A practical state transition can be specified as follows:

```
certify at current interval, center, and enclosure precision;
if successful:
    tighten the error using the root intersection;
    retain the best proved certificate so far;
    if the requested mixed tolerance is proved: return;
    choose a new center inside the proved root enclosure;
if arithmetic uncertainty dominates or progress has stalled:
    increase enclosure order / significant bits;
if useful and allowed:
    refine the asymptotic seed;
stop with the best certificate and the reason for noncompletion;
```

The model’s doubling-after-every-miss policy is deliberately simpler than this proposal. Its purpose is to demonstrate that the required accuracy is recoverable by changing the correct resource, not to prescribe optimal production scheduling.

For mixed absolute and relative accuracy, a proved lower bound $m \leq |x_*|$ gives a sufficient absolute threshold

$$\tau = \max(\tau_{\text{abs}}, \tau_{\text{rel}}m).$$

The initial precision planner can use this threshold once $m > 0$ is available. Before such a magnitude bound exists, it should retain the relative demand as an unresolved precision requirement rather than silently omit it. The existing safeguards concerning relative accuracy at a zero root should remain intact.[10]

The best-certificate store should compare proved bounds, not merely replace the previous result with the last association. The source currently records the latest successful attempt as `best`; there is no ordering test. A future enclosure-order change or seed replay need not monotonically improve the center-error bound, so retaining the best by an explicit criterion makes the failure result more useful without changing the proof system.

5.6 Why the existing tests do not exercise this case

The inspected acceptance tests sweep absolute and relative tolerances through 10^{-20} with a generously chosen enclosure order. Their explicit arithmetic-precision recovery test uses a *fixed* center and an initial derivative-separation failure, which selects the controller branch that already doubles the order. They do not test a long sequence of successful but insufficient automatic-center certificates.[27]

Add a deliberately low-order, automatic-center, tight-tolerance test; a relative-only high-accuracy test; and an assertion that the public error bound is no weaker than the bound computed from the retained root enclosure. Separate the statement “a certificate was obtained” from “the requested accuracy was reached” in both assertions and diagnostics. This is new coverage of a specific state-machine defect, not a repetition of the earlier general requests for better certificates or a nonempty native test gate.

6 F03: flat products forget the dominant exponential grade

Classification: medium-priority precision and performance issue. The current bound is conservative in the example below. The defect is avoidable loss of algebraic precision caused by discarding exponential grading *before* deciding which error term dominates. The same implementation also computes full coefficient products in sectors that will not be retained.

Source locus. `AsymptoticInverse/Kernel/FlatSectorOperations.wl`: `flatOpsMultiplyData`, approximately lines 90–119; `flatOpsJetBound`, `flatOpsBoundProduct`, and `flatOpsMake`. [16].

6.1 Two independent truncations must remain independent

The flat representation uses

$$s(u) = \sum_{k=0}^N \mathcal{E}(u)^k A_k(u) + \mathcal{E}(u)^{N+1} \mathcal{O}(u^p L^d), \quad \mathcal{E}(u) = e^{-\lambda/u^\nu}, \quad L = 1 + |\log u|, \quad (10)$$

with fixed $\lambda, \nu > 0$. Positive-sector amplitudes can themselves have power-log remainders; the zero sector is exact. The package stores those inner errors separately from the complete omitted-sector tail.[15, 16]

For two inputs of depths N_a, N_b , the product retains depth $N = \min(N_a, N_b)$. The implementation constructs the complete convolution through degree $N_a + N_b$ using `pMul` and `pAdd`. It then combines all discarded finite coefficients, all input-tail/finite-sector products, and the tail/tail product into a single algebraic pair (p, d) . The exponential degree of each discarded contribution is not retained in this comparison. Finally, `flatOpsMake` multiplies the combined pair by $\mathcal{E}^{N+1}$. [16]

The source comment justifies each individual weakening using $\mathcal{E}^{k-N-1} \leq 1$ for $k > N$. That inequality is true. But applying it independently to every term can let a very remote sector with a large algebraic pole determine the bound for the entire omitted tail. The later exponential should first be compared against the earlier exponential, because it dominates every fixed algebraic/logarithmic disadvantage.

6.2 A one-sector example loses three algebraic orders

Take the inverse of

$$y = x + e^{-1/x}, \quad x \rightarrow 0^+,$$

and write $u = y$, $\mathcal{E} = e^{-1/u}$. Lagrange inversion in the exponential marker gives

$$x(u) = u - \mathcal{E} + \frac{\mathcal{E}^2}{u^2} + \mathcal{E}^3 \left(\frac{1}{u^3} - \frac{3}{2u^4} \right) + \cdots . \quad (11)$$

A depth-one object therefore carries

$$s = u - \mathcal{E} + \mathcal{E}^2 \mathcal{O}(u^{-2}). \quad (12)$$

Squaring the represented function gives

$$s^2 = u^2 - 2u\mathcal{E} + \mathcal{E}^2 + \mathcal{E}^2 \mathcal{O}(u^{-1}) + \mathcal{E}^3 \mathcal{O}(u^{-2}) + \mathcal{E}^4 \mathcal{O}(u^{-4}) \quad (13)$$

$$= u^2 - 2u\mathcal{E} + \mathcal{E}^2 \mathcal{O}(u^{-1}). \quad (14)$$

Indeed, the first omitted exact coefficient from (11) is $1 + 2/u$, so the u^{-1} bound is not an arbitrary optimistic guess.

The inspected implementation combines the algebraic exponents $0, -1, -2, -4$ after suppressing their distinct exponential degrees. The tail/tail product then wins and the result carries

$$s^2 = u^2 - 2u\mathcal{E} + \mathcal{E}^2 \mathcal{O}(u^{-4}), \quad (15)$$

three algebraic orders weaker than (14). Both are valid asymptotic bounds; only the first uses the available grading correctly.

The supplied native characterization separates baseline and proposal outputs:

```
s = AsymptoticFlatInverse[x + Exp[-1/x], {x, 0}, {y, 1}];
a = FlatSeriesMultiply[s, s];
b = AsymptoticAudit[FlatSeriesMultiplyGraded[s, s];
{a["SectorRemainder"], b["SectorRemainder"]}]
```

The source-derived expected inner tail powers are -4 and -1 , respectively. This call was not executed natively in the audit.

6.3 The loss grows with retained depth

The coefficient of $\mathcal{E}^k$ in the flat inverse is

$$c_k(u) = \frac{(-1)^k}{k!} \left(\frac{d}{du} + \frac{k}{u^2} \right)^{k-1} 1. \quad (16)$$

This follows by applying the Lagrange–Bürmann derivative formula to the marker equation $x = u - ze^{-1/x}$ and then setting $z = 1$ after grouping the exponential factors. Equivalently, each derivative of $P(u)e^{-k/u}$ is

$$e^{-k/u} \left(P'(u) + \frac{k}{u^2} P(u) \right).$$

The strongest pole in (16) is obtained by choosing the phase-derivative contribution at every step:

$$c_k(u) = \frac{(-1)^k k^{k-1}}{k!} u^{-2(k-1)} + \text{less singular powers}. \quad (17)$$

The exact sparse-Laurent implementation in `audit_models.py` checks the first coefficients and this leading term for a range of k .

For a depth- N inverse, the inherited tail is $\mathcal{E}^{N+1}\mathcal{O}(u^{-2N})$. The zero sector is u , so multiplication by the other zero sector yields the dominant square error

$$\mathcal{E}^{N+1}\mathcal{O}(u^{1-2N}). \quad (18)$$

Finite products on the first omitted sector have algebraic power $2 - 2N$, which is smaller in magnitude than the inherited u^{1-2N} term. The tail/tail product lies at sector $2N + 2$, but the baseline flattens it to sector $N + 1$ and lets its algebraic power $-4N$ dominate. The resulting algebraic precision loss is

$$(1 - 2N) - (-4N) = 2N + 1. \quad (19)$$

Depth N	Baseline power	Graded power	Orders lost	Full / retained coefficient pairs
1	-4	-1	3	4/3
2	-8	-3	5	9/6
3	-12	-5	7	16/10
5	-20	-9	11	36/21
8	-32	-15	17	81/45

These are exact support/envelope-model calculations, not measured native performance figures. The baseline power formula is the consequence of the inspected tail-combination order for this family.

6.4 A graded error semiring sufficient for this repair

Represent an individual bound by a triple

$$(k, \beta, d) \longleftrightarrow \mathcal{E}^k \mathcal{O}(u^\beta L^d).$$

Products add triples componentwise. Dominance is lexicographic in $(k, \beta, -d)$: smaller exponential degree first, then smaller algebraic power, then larger logarithmic degree. No general transseries engine is needed for this restricted single-phase calculation.

Lemma 2. *For fixed $\lambda, \nu > 0$, fixed real β_1, β_2 , and fixed logarithmic degrees, every sector $k_2 > k_1$ is negligible relative to sector k_1 :*

$$\frac{\mathcal{E}^{k_2} u^{\beta_2} L^{d_2}}{\mathcal{E}^{k_1} u^{\beta_1} L^{d_1}} \longrightarrow 0 \quad (u \rightarrow 0^+).$$

Proof. The logarithm of the ratio is

$$-\frac{(k_2 - k_1)\lambda}{u^\nu} + (\beta_2 - \beta_1) \log u + (d_2 - d_1) \log L.$$

The first term tends to $-\infty$ faster in magnitude than the other two. Exponentiating proves the result. $\square$

The finite list of tail candidates for a product must retain these grades:

$$\begin{aligned} (i + j, B(A_i) + B(B_j)), & \quad i + j > N, \\ (N_a + 1 + j, B(R_a) + B(B_j)), & \quad 0 \leq j \leq N_b, \\ (N_b + 1 + i, B(R_b) + B(A_i)), & \quad 0 \leq i \leq N_a, \\ (N_a + N_b + 2, B(R_a) + B(R_b)). & \end{aligned}$$

Here B abbreviates an algebraic/logarithmic bound pair, addition of pairs means componentwise addition, and a coefficient's bound includes its unknown inner remainder. Exact zero contributes no candidate. The dominant triple is selected before any sector is weakened.

For compatibility with the existing result schema, a dominant grade (K, β, d) with $K \geq N + 1$ can then be weakened to the legacy $\mathcal{E}^{N+1}\mathcal{O}(u^\beta L^d)$. Crucially, this happens *after* dominance selection. The schema need not acquire an arbitrary transseries support merely to fix the present loss. A later enhancement can preserve K explicitly when all earlier omitted sectors vanish.

This ordering also explains why unrelated unknown errors must not be canceled. Equality of two finite approximations does not establish equality of their error functions. The proposal combines envelopes by dominance, never by signed cancellation. It preserves the existing separate inner remainders for retained positive sectors.

6.5 Avoid computing discarded coefficient polynomials

Only pairs $i + j \leq N$ contribute retained coefficient polynomials. Pairs above that diagonal are needed only as bounds unless the implementation deliberately wants to discover cancellations in an omitted frontier. For equal depth N , the number of retained pairs is

$$\sum_{i=0}^N (N - i + 1) = \frac{(N + 1)(N + 2)}{2},$$

compared with $(N + 1)^2$ pairs in the complete convolution. The tail bound of a discarded pair can be formed from the two coefficient envelopes without multiplying and canonicalizing their entire logarithmic polynomials.

This removes nearly half of the expensive coefficient-pair operations at large equal depths; it does not change the quadratic order of a naive scan over all candidate pairs. Nor does it guarantee a factor-of-two runtime improvement: coefficient sizes, cancellations, and symbolic simplification dominate real costs. The supplied proposal preserves the baseline's aggregate pair-budget check, avoiding an unreviewed change in resource-policy semantics while reducing actual coefficient work.

There is a precision tradeoff worth making explicit. The current full convolution can discover exact cancellations among discarded finite coefficients. An envelope-only scan deliberately does not pay for those cancellations and therefore is not guaranteed to improve every possible input's tail. It improves the exhibited inverse family and prevents remote sectors from artificially controlling earlier ones, but a carefully arranged finite polynomial can benefit from omitted-frontier cancellation. A hybrid implementation can aggregate only the potentially dominant omitted sector, or only its leading algebraic/logarithmic block, under a separate small budget. Unknown remainders must still remain uncanceled.

6.6 The delivered candidate and its validation boundary

`FlatSeriesMultiplyGraded` in `proposals/AuditSupport.wl` is a named alternative, not an override of the upstream operation. It reuses the pinned private alignment, coefficient-bound, sparse precision arithmetic, truncation, and object-construction helpers. The implementation changes the scheduling and envelope grading while leaving retained inner-error transport to the existing primitives. Its returned flat result also receives the explicit target approach from the `F01` helper.

Because private helpers are used, the candidate is revision-specific. Its custom operation recipe has no new replay/refinement implementation in this archive; it is a candidate product and error-transport

operation, not a complete integration into every recipe consumer. It was not loaded in a native kernel. The archive provides focused desired-contract and proposal-contract tests; none are advertised as a completed native test run. The Python model separately tests the mathematical graded bound and selected exact coefficients, not the full Wolfram candidate.

The inspected upstream multiplication test compares the finite product with a marker-polynomial oracle and checks that a sector remainder is nonzero. That cannot distinguish (14) from (15); both have the same retained polynomial and a nonzero tail. New assertions must inspect the tail power, inner-remainder preservation, unequal input depths, exact-zero annihilation, and products where omitted finite coefficients cancel.[26]

7 F04: a positive observable can have a negative source

Classification: conservative capability restriction and API inconsistency. This finding identifies a useful safe generalization, not an unsafe existing result.

Source locus. `AsymptoticInverse/Kernel/GammaInverseOperations.wl: gammaInverseSeriesPower`, especially the early endpoint/noninteger-exponent guard and the later source-equation replay.[21].

The Gamma/Barnes power operation rejects every noninteger outer exponent when the recorded source endpoint is not $+\infty$. The diagnostic describes this as requiring a positive source branch. However, the input object can already represent a *power observable* of the source, and positivity of that observable is the relevant requirement for a real noninteger outer power.

For example, let $x < 0$ be the selected source and let the input object represent $s = x^2$. Then

$$s > 0, \quad s^{1/2} = \sqrt{x^2} = |x| = -x.$$

The source is negative but the square-root input is positive. A characterization case is

```
s = AsymptoticInverse[
  Gamma[-x], {x, -Infinity}, {y, 2}, "Power" -> 2];
SeriesPower[s, 1/2]
```

The runner first exposes the resulting object's power and endpoint, then calls the power operation. The inspected guard rejects the noninteger outer exponent for a Gamma/Barnes inverse object with that negative endpoint. Exact public constructor behavior is left to the native characterization; the guard's policy is explicit in source.[19, 21]

The public arithmetic wrapper does not rescue this rejection through its composite fallback: **UnsupportedPowerBranch** is absent from `seriesArithmeticFallbackQ`. Consequently, once the input is the specified Gamma/Barnes inverse object, the specialized guard is propagated as a failure rather than replaced by a positive-observable envelope calculation.[12, 11, 21]

7.1 Do not repair this by multiplying exponents blindly

The tempting rewrite $(x^r)^k \mapsto x^{rk}$ is incorrect on a negative source branch. The example $r = 2, k = 1/2$ would yield x instead of $-x$. A safe extension uses a positive source coordinate $u = -x > 0$:

$$f(x) = y \iff f(-u) = y, \quad (x^{2m})^k = u^{2mk}.$$

The source condition must be reflected as well. The desired result can be reconstructed from an inverse problem at $u \rightarrow +\infty$, with an explicit observable map back to the original source.

This reflection also makes the distinction between the original source and the represented observable visible in metadata. A result cannot claim to represent the original x^{rk} on a negative branch when it actually represents $|x|^{rk}$. Either record the observable expression explicitly or retain a sign-correct reconstruction recipe. Merely bypassing the guard is not a repair.

7.2 Inherited precision remains a constraint on source replay

If $s = A + \mathcal{O}(R)$ has leading power α in its positive small coordinate, a fixed real power k on an established branch transports the absolute error at first order as

$$\delta(s^k) = \mathcal{O}(A^{k-1}R), \quad P_{\text{out}} \leq P_{\text{in}} + \alpha(k-1).$$

The existing Gamma/Barnes replay operation already caps output precision using this relation and carries a reciprocal-log correction to the tail. That cap must survive a sign-aware extension. Replaying the exact source equation to obtain coefficients does not justify pretending the input object was supplied with greater accuracy.[21]

A minimal development target is therefore narrow: accept positive even source-power observables on negative affine Gamma/Barnes branches, reflect to a positive source coordinate, and preserve the existing precision cap. General noninteger powers of negative observables remain a different branch-analysis problem and should not be smuggled into this change.

8 A focused development plan, without duplicating the old roadmap

The earlier register already contains broad initiatives for scale protocols, more certified functions, general transseries, parameter regions, release gates, and documentation. The plan below is deliberately limited to the concrete new invariants and algorithms established in this report.[3]

8.1 Change set A: establish target geometry before arithmetic

Patch the flat constructor and derived-flat constructor to record the exact approach of their monomial target chart. Patch Erfc-tail and quadratic-threshold adapters to compute direction from the actual oriented target map. Update legacy fallback so that a guessed offset or affine scale cannot bypass the positive-coordinate limit check.

The acceptance criterion is not merely that a metadata key exists. Original and derived objects must agree on endpoint and side, their positive coordinates must tend to zero there, and mixed-scale public arithmetic must either preserve that approach or reject genuinely incompatible approaches. The sixteen exact model cases provide a small orientation matrix; add producer/fallback parity tests for each applicable public family.

This change should come first because incorrect independent-variable geometry can undermine the interpretation of every later error calculation. It is localized enough to avoid a broad object-schema redesign.

8.2 Change set B: separate certification from accuracy completion

Use the existing root intersection to tighten the reported center-error bound. Retain the best successful certificate by an explicit bound criterion. Then repair the automatic-center transition so that insufficient arithmetic precision can increase after successful certification. Add relative-only demand to the precision planner once a root-magnitude bound is available.

Test the low-order $x^2 = 2$ case with source refinement disabled so that no expensive symbolic seed replay masks the controller behavior. Require the final certificate to establish the tolerance, and require its arithmetic history to show an appropriate response when low precision is the bottleneck. Also test a fixed center whose actual error exceeds the requested tolerance: the controller must still report an accuracy floor rather than silently moving it.

The source of every returned bound remains unchanged: a root of the stored explicit equation on a verified interval. This change does not certify an unspecified `InputRemainder` family, and it should not alter that existing scope statement.[10]

8.3 Change set C: retain grades in flat product error scheduling

Introduce a small internal error-grade triple and use it only for the complete omitted-sector tail. Keep retained inner errors in their own sectors. Replace discarded coefficient products by envelope products, with optional dominant-frontier cancellation work when its cost is justified. Preserve current aggregate resource guards initially; separate changes to budget policy are already tracked elsewhere.

The decisive precision acceptance tests are the depth-one and depth-three squares in F03, whose complete tail powers should be -1 and -5 , not -4 and -12 . The decisive safety tests are exact-zero annihilation, pure unknown inner coefficients, unequal sector depths, and derivative-contract preservation. The decisive performance instrumentation counts retained polynomial multiplications separately from envelope comparisons; wall-clock figures alone would not explain the benefit.

8.4 Change set D: support reflected positive observables

After geometry and precision transport are stable, extend Gamma/Barnes observable replay to the positive-even-power case described in F04. Keep source reflection and observable reconstruction separate. Test $\sqrt{x^2} = -x$ for $x < 0$, not just the equality of formal exponent products. Compare a directly requested positive-source observable with the reflected-source operation under the same cutoff and inherited remainder.

This is a feature-sized change, not a prerequisite for correcting F01–F03. A deliberate continued rejection is mathematically safer than a branch-blind implementation.

8.5 New cross-family invariants worth institutionalizing

The new defects share a theme: locally sensible metadata shortcuts become wrong when an object enters a different algebraic path. Three targeted metamorphic properties expose these errors better than another large list of unrelated examples.

Chart naturality. Construct an expansion, apply an exact affine target change, and compare the inferred target approach with the transformed original approach. Source reflection must be handled independently of target scaling.

Precision monotonicity under informative post-processing. A certificate which retains a smaller root enclosure must not report an error upper bound larger than the maximum distance from its center to that enclosure. This is a theorem about already retained evidence, not a heuristic request for tighter numerics.

Graded dominance. Replacing a tail contribution by a later exponential sector cannot worsen the leading exponential/algebraic order of an earlier tail merely because its coefficient has a larger fixed pole. Test the grade comparison directly, as well as through public multiplication.

These invariants are concrete additions to existing tests, not a restatement of the general need for more tests. They each correspond to a derived equation in this report and can be implemented with small exact oracles.

9 Overall assessment

The repository’s strongest contribution is the connection between structured asymptotic expressions and explicit operational contracts: source charts, real branches, tracked power–log errors, qualified differentiation, exact-core inversion, and quantitative residual certificates. Its special-function work correctly leans on native Wolfram facilities rather than trying to reproduce their entire domain knowledge.[2, 9, 23, 10]

The additional issues found here concentrate at the boundaries between those contracts. A target offset is substituted for a target limit; success of one proof obligation suppresses progress on another; an exponentially graded uncertainty is flattened before dominance is decided; the sign of a source is substituted for the sign of an observable. Each shortcut has a small local justification, but the larger mathematical object requires a stronger invariant.

The recommended response is correspondingly focused: preserve exact target geometry, let certificate precision respond to the actual bottleneck, preserve exponential grades until error dominance is resolved, and distinguish source orientation from observable positivity. These changes are more valuable at this stage than expanding the feature list while leaving the existing cross-family semantics implicit.

A Reproducibility and delivered artifacts

A.1 Executed evidence

The independent checks use Python 3.10+ syntax and only the standard library. The recorded run used Python 3.13.5. From the extracted archive root:

```
python -m unittest discover -s tests -p 'test_models.py' -v
python code/generate_evidence.py
```

The first command ran twenty tests successfully. The second regenerates `evidence/independent_results.json`, including the sixteen orientation cases, both certificate-policy histories, and the flat-square depth table. Preserve the delivered evidence before rerunning when run provenance matters.

`code/audit_models.py` is deliberately restricted. It is not a Wolfram interpreter, a substitute implementation of `AsymptoticInverse`, or a complete interval library. Its certificate model specializes to a positive square root and independently verifies every retained interval by squaring its rational endpoints. Its flat model represents integer Laurent powers for the selected inverse family. Those

restrictions are strengths for checking the stated mechanisms, but they do not validate arbitrary package inputs.

A.2 Native experiments supplied but not executed

Set the package path to a local reviewed copy of the pinned standalone file or to the modular kernel entry in a complete pinned checkout. No supplied script downloads or executes remote code automatically.

```
# POSIX shell
export ASYMPOTIC_AUDIT_PACKAGE=/absolute/path/AsymptoticInverse.wl
wolframscript -file code/Characterize.wls
wolframscript -file code/CompareBuiltins.wls

# PowerShell
$env:ASYMPOTIC_AUDIT_PACKAGE =
  (Resolve-Path 'C:\path\AsymptoticInverse.wl').Path
wolframscript -file code/Characterize.wls
```

The characterization runner records nine bounded probes, including private approach inference and public fallback behavior separately. The comparison runner records twelve native/package calls. Their elapsed times, when the user runs them, are individual in-process call timings, not a controlled speed benchmark. Matching order contracts and repeated fresh-kernel runs are necessary before drawing performance conclusions.

The runners record the actual native version, platform, and entry-file hash. An entry-file hash does not attest modular companion files, and the recorded expected commit is not proof that an arbitrary supplied path has that commit. The scripts explicitly mark this distinction. Each ordinary probe has a 45-second time limit and a 512 MiB additional-allocation limit; load and test-suite limits are separate. A runner limit is an audit-harness outcome, not proof of a package algorithmic failure.

To run the proposal tests or desired upstream contracts:

```
# POSIX shell; Proposal is also the default
export ASYMPOTIC_AUDIT_SUITE=Proposal
wolframscript -file code/RunNativeTests.wls
export ASYMPOTIC_AUDIT_SUITE=Desired
wolframscript -file code/RunNativeTests.wls

# PowerShell
$env:ASYMPOTIC_AUDIT_SUITE = 'Proposal'
wolframscript -file code/RunNativeTests.wls
```

The proposal suite contains seven tests; the desired-contract suite contains eight. The latter intentionally includes behavior expected to fail on the baseline, and F04 remains a feature proposal. The runner rejects an empty/noncompleted suite and uses the documented report-success property. Exit status is zero for a successful nonempty suite, one for test failures, and two for loading, runner, or noncompletion errors.[38]

The Wolfram runners and helpers have not been executed or certified as syntactically accepted by a native kernel in this environment. Their results are not present in the delivered evidence directory. The output functions refuse to overwrite prior native evidence files.

A.3 Proposal scope

Artifact	Scope and limitations
<code>AuditSupport.wl</code>	New namespaced helpers only. Explicit target geometry for selected charts; tightening of an existing certificate; revision-specific graded flat multiplication using private upstream primitives. No global overrides.
<code>audit_models.py</code>	Executed restricted mathematical models. No package loading and no claim to emulate general Wolfram evaluation semantics.
<code>DesiredContracts.wlt</code>	Eight intended upstream contracts, including a future F04 capability. Not a baseline passing suite.
<code>ProposalContracts.wlt</code>	Seven focused tests for the supplied helpers. Not executed natively. Not a full regression suite.
<code>Characterize.wls</code>	Records source-predicted behavior without rewriting it into a claimed observation.
<code>CompareBuiltin.wls</code>	Paired result characterization under explicit resource limits; not matched-precision benchmarking.

No remote repository was changed. No full upstream source snapshot or font files are included. The maintained source should remain the canonical modular kernel; a reviewed production change would require regeneration of the standalone distribution and native regression testing. This operational note is not claimed as a new finding.

A.4 Rebuilding the article

The article uses standard LaTeX packages and Latin Modern through the installed TeX distribution. No font binaries are distributed. From `article/`:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

The bibliography is an included conventional reference list, so a separate BibTeX step is unnecessary. The complete editable source consists of the master, the six section files, and `references.tex`. Rendered pages were checked for layout defects in the delivered PDF; the evidence directory contains the build/layout summary.

B Coverage and novelty ledger

B.1 Direct source coverage

The file-level manifest gives source windows and available Git blob identifiers. The directly inspected implementation families were: the core constructor and sparse precision primitives; ordinary series operations and automatic arithmetic; source and target coordinate reconstruction; exact inverse certificates and numerical checks; flat construction and operations; native special-function import and composite arithmetic; Dirichlet special functions; Gamma and Barnes inverse construction;

Gamma/Barnes observable power replay; and Fourier coefficients. Some files were read in full and others in selected windows. Unread middle ranges are not treated as verified.

`FlatSectorOperations.wlt` and `AcceptanceCertificates.wlt` were inspected in full for the new product and controller cases. Repository test records and prior review execution records were read as historical evidence where relevant, but their reported native successes were not reproduced in this audit. No full native suite result is attributed to this work.

B.2 Relationship to earlier review entries

Finding	Related existing topic	Why the present contribution is additional
F01	D01/D02: coordinate contracts and a common scale protocol.	Identifies the exact offset/scale-priority defect, three constructor families with wrong approaches, exact orientation formulas, and public/private regression boundaries. It does not merely request a protocol.
F02	X05: improved quantitative certificates; C08: ordinary recipe-refinement precision.	Analyzes a different controller: successful certificate attempts freeze enclosure order. Supplies a rational reproducer and a theorem for a tighter error already implied by the retained interval. No restatement of fixed $h + 2$ series replay.
F03	P01/P06: eager computation and resource semantics; X08: broader transseries.	Identifies loss of single-phase exponential grading, derives the exact $2N + 1$ -order degradation for a supported family, and proposes a restricted graded-tail algorithm. No multirate transseries redesign is required.
F04	X02: specialized coefficient-algebra capabilities.	Isolates a source-sign/observable-sign mismatch and a branch-correct reflected replay for positive even-power observables. It is not a generic request to unify Gamma arithmetic.

The relationships above are based on the consolidated register and inspected review summaries, not a claim that no sentence anywhere in an uninspected prior article could resemble a present observation. The machine-readable findings ledger records the same qualification.[3, 5, 6, 7, 8]

B.3 Remaining uncertainty

The strongest source deductions are the explicit approach-priority logic, the enclosure-order branch, and the flat-tail grade erasure. The main uncertainty is their precise behavior through all public wrappers on a native Wolfram 15 kernel, including symbolic simplification, dispatch, messages, and performance. The probes are designed to make that distinction visible.

The proposed helpers are not a completed repair of all four findings: F01 still needs producer integration; F02's supplied helper tightens existing bounds while the controller redesign remains a specified change; F03 has a named alternative implementation requiring native verification; F04 has a sign-correct design and desired behavior test, not a completed Gamma/Barnes reflection adapter. These boundaries are recorded so that the archive can serve as a development input without being mistaken for a validated replacement release.

References

- [1] Asymptotic Contributors. Pinned repository snapshot. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/tree/921387e5ba1239bfda96e63e64e89bf63d9c41e6>.
- [2] Asymptotic Contributors. Repository README. README.md. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/README.md>.
- [3] Asymptotic Contributors. Consolidated code-review status. docs/development/CODE_REVIEW_STATUS.md. Pinned snapshot 921387e. https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/docs/development/CODE_REVIEW_STATUS.md.
- [4] Asymptotic Contributors. Existing code-review index. code-review/README.md. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/README.md>.
- [5] Asymptotic Contributors. Review 6: accompanying audit summary. code-review/code-review-6/README.md. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-6/README.md>.
- [6] Asymptotic Contributors. Review 7: accompanying audit summary. code-review/code-review-7/README.md. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-7/README.md>.
- [7] Asymptotic Contributors. Review 8: accompanying audit summary. code-review/code-review-8/README.md. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-8/README.md>.
- [8] Asymptotic Contributors. Review 9: accompanying audit summary. code-review/code-review-9/README.md. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/code-review/code-review-9/README.md>.
- [9] Asymptotic Contributors. Core constructor and precision primitives. AsymptoticInverse/Kernel/AsymptoticInverse.wl. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/AsymptoticInverse.wl>.
- [10] Asymptotic Contributors. Exact inverse certificate implementation. AsymptoticInverse/Kernel/InverseCertificates.wl. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/InverseCertificates.wl>.
- [11] Asymptotic Contributors. Ordinary series operations. AsymptoticInverse/Kernel/SeriesOperations.wl. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SeriesOperations.wl>.

- `//github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SeriesOperations.wl.`
- [12] Asymptotic Contributors. Automatic arithmetic and held normalization.
`AsymptoticInverse/Kernel/SeriesArithmetic.wl.` Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SeriesArithmetic.wl>.
 - [13] Asymptotic Contributors. Logarithmic target-coordinate inversion.
`AsymptoticInverse/Kernel/CoordinateInverse.wl.` Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/CoordinateInverse.wl>.
 - [14] Asymptotic Contributors. Source-coordinate reconstruction.
`AsymptoticInverse/Kernel/SourceCoordinates.wl.` Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SourceCoordinates.wl>.
 - [15] Asymptotic Contributors. Flat inverse construction.
`AsymptoticInverse/Kernel/FlatSectors.wl.` Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/FlatSectors.wl>.
 - [16] Asymptotic Contributors. Flat-sector operations.
`AsymptoticInverse/Kernel/FlatSectorOperations.wl.` Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/FlatSectorOperations.wl>.
 - [17] Asymptotic Contributors. Composite arithmetic and target-approach inference.
`AsymptoticInverse/Kernel/SeriesEnvelopeArithmetic.wl.` Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SeriesEnvelopeArithmetic.wl>.
 - [18] Asymptotic Contributors. Special inverse adapters.
`AsymptoticInverse/Kernel/SpecialFunctionAdapters.wl.` Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/SpecialFunctionAdapters.wl>.
 - [19] Asymptotic Contributors. Gamma inverse construction.
`AsymptoticInverse/Kernel/GammaInverse.wl.` Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/GammaInverse.wl>.
 - [20] Asymptotic Contributors. Barnes inverse construction.
`AsymptoticInverse/Kernel/BarnesInverse.wl.` Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/BarnesInverse.wl>.
 - [21] Asymptotic Contributors. Gamma/Barnes observable powers.
`AsymptoticInverse/Kernel/GammaInverseOperations.wl.` Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/GammaInverseOperations.wl>.

- [22] Asymptotic Contributors. Fourier-polynomial coefficients.
AsymptoticInverse/Kernel/FourierCoefficients.wl. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/FourierCoefficients.wl>.
- [23] Asymptotic Contributors. Native special-function import.
AsymptoticInverse/Kernel/NativeSpecialFunctions.wl. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/NativeSpecialFunctions.wl>.
- [24] Asymptotic Contributors. Dirichlet special-function adapters.
AsymptoticInverse/Kernel/DirichletSpecialFunctions.wl. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/DirichletSpecialFunctions.wl>.
- [25] Asymptotic Contributors. Numerical inverse checks.
AsymptoticInverse/Kernel/NumericalInverseChecks.wl. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Kernel/NumericalInverseChecks.wl>.
- [26] Asymptotic Contributors. Flat-sector operation tests.
AsymptoticInverse/Tests/FlatSectorOperations.wlt. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Tests/FlatSectorOperations.wlt>.
- [27] Asymptotic Contributors. Certificate acceptance tests.
AsymptoticInverse/Tests/AcceptanceCertificates.wlt. Pinned snapshot 921387e. <https://github.com/VladimirReshetnikov/Asymptotic/blob/921387e5ba1239bfda96e63e64e89bf63d9c41e6/AsymptoticInverse/Tests/AcceptanceCertificates.wlt>.
- [28] Wolfram Research. *Asymptotic*, Wolfram Language documentation. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [29] Wolfram Research. *AsymptoticSolve*, Wolfram Language documentation. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [30] Wolfram Research. *InverseSeries*, Wolfram Language documentation. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [31] Wolfram Research. *ComposeSeries*, Wolfram Language documentation. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/ComposeSeries.html>.
- [32] Wolfram Research. *SeriesData*, Wolfram Language documentation. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [33] Wolfram Research. *Series*, Wolfram Language documentation. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Series.html>.
- [34] Wolfram Research. *ProductLog*, Wolfram Language documentation. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/ProductLog.html>.
- [35] Wolfram Research. *Interval*, Wolfram Language documentation. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Interval.html>.

- [36] Wolfram Research. *AsymptoticIntegrate*, Wolfram Language documentation. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticIntegrate.html>.
- [37] Wolfram Research. *AsymptoticSum*, Wolfram Language documentation. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticSum.html>.
- [38] Wolfram Research. *TestReportObject*, Wolfram Language documentation. Accessed 9 September 2026. <https://reference.wolfram.com/language/ref/TestReportObject.html>.
- [39] NIST Digital Library of Mathematical Functions. DLMF 2.1: Definitions and elementary properties of asymptotic approximations. Accessed 9 September 2026. <https://dlmf.nist.gov/2.1>.
- [40] NIST Digital Library of Mathematical Functions. DLMF 5.11: Asymptotic expansions of the Gamma function. Accessed 9 September 2026. <https://dlmf.nist.gov/5.11>.
- [41] NIST Digital Library of Mathematical Functions. DLMF 5.17: Barnes G-function. Accessed 9 September 2026. <https://dlmf.nist.gov/5.17>.
- [42] NIST Digital Library of Mathematical Functions. DLMF 7.12: Error-function asymptotic expansions. Accessed 9 September 2026. <https://dlmf.nist.gov/7.12>.